# TEMA 2. EL MODELO DE OBJETOS DE DOCUMENTO DOM

## *Introducción al DOM*

Como desarrolladores, el acceso al DOM mediante JavaScript nos permite:

- crear documentos HTML
- explorar su estructura, y
- añadir/eliminar/modificar los elementos que los componen.

El navegador se encarga de mantener **sincronizados** el documento y DOM.

¿Qué implica que el documento y DOM estén sincronizados?

- Cualquier modificación que realicemos en el DOM a través de JavaScript se trasladará automáticamente a la presentación del documento;

- Ejemplo: cambiar un estilo, o borrar un nodo.

El DOM **es una especificación genérica** que se aplica tanto a:

- documentos HTML
- como a documentos XML,
- y que no está ligada en exclusiva a ningún lenguaje de programación.

## *Niveles de DOM y compatibilidad*

DOM Level 0

En 1995 se había iniciado el desarrollo de JavaScript.

En 1996 Netscape Navigator incluía una versión del DOM limitada, denominada nivel 0, y que básicamente sólo servía para:

- validar formularios y
- crear efectos rollover

1998 estandarización de JavaScript en la forma de ECMAScript.

DOM level 1

Finales de 1998 y durante 1999 el W3C toma cartas en la estandarización del DOM.

El W3C por fin publica la primera especificación del DOM, que se denominaría nivel 1, y a partir de ese momento la labor de los desarrolladores web se facilitó enormemente.

Hasta el 1999 Netscape y Microsoft habían mantenido una lucha por la prevalencia de sus navegadores que habían alcanzado ya la versión 4.

A partir de este momento todos los navegadores empiezan a competir por lograr el mayor grado de compatibilidad con esta especificación, en lugar de seguir haciendo la guerra por su cuenta intentando deslumbrar a los usuarios con funcionalidades que sólo estaban disponibles en sus productos y que los demás fabricantes se negaban a adoptar.

DOM level 2

Finales del 2000.

DOM level 3

Año 2004.

Observaciones:

El grado de compatibilidad de todos los navegadores con estas especificaciones es actualmente muy alto.

- Persisten algunos problemas de interpretación provocados porque el W3C no especifica exclusivamente el DOM:
  - Hay que tener en cuenta que se utiliza fundamentalmente para páginas web y con JavaScript.
- sino que el W3C lo especifica con un nivel de abstracción superior que permite aplicarlo también a documentos XML
- y con cualquier lenguaje de programación.

Afortunadamente, las bibliotecas JavaScript como **jQuery** se encargan de homogeneizar estar discrepancias, liberando al programador de tener que seguir preocupándose por ellas.

Actualmente (2012) se está desarrollando la especificación del DOM 4, que aún se encuentra en fase de borrador.

https://developer.mozilla.org/es/docs/Los_niveles_del_DOM

## *Tipos de datos específicos del DOM (Data Types)*

Utilizar el DOM se reduce en última instancia al manejo de objetos que nos ofrecen propiedades y métodos.

Los valores de estas propiedades y las respuestas de los métodos pueden ser:

- de los tipos intrínsecos de JavaScript que repasamos en el tema anterior (String, Number, …),
- bien **otros objetos cuya estructura (interfaz) está estandarizada en el DOM**. Estas estructuras pueden considerarse como tipos de datos específicos del DOM.

Los Tipos Específicos del DOM más usuales son :

(Consultar */en-US/docs/Web/API/Document_Object_Model/Introduction)

- **Element**. Es un tipo de datos que hace referencia a un nodo de tipo elemento (los **nodos de tipo elemento** son los que corresponden a un elemento HTML, como `p` o `form`).

  Existen otros dos tipos de datos íntimamente relacionados con éste:

  - por un lado tenemos uno más genérico llamado **`node`** que hace referencia a un nodo sea cual sea su tipo. **element** hereda de **node.**

- ❖ por otro lado tenemos uno más concreto llamado **HTMLElement** que hace referencia a nodos de tipo elemento HTML (no XML). **HTMLElement** hereda de **element**.

- **NodeList**. Es una colección **ordenada** de objetos de tipo nodo. Una colección nos permite acceder a sus elementos como si fuera un array, es decir, indicando el índice entre corchetes, pero **no admite** ciertos métodos que sí están disponibles en los arrays.

  Cada element en un **nodeList** es accessible por su índice in either of two ways:

  - ❖ list.item(1): this is the single method on the nodeListobject.
  - ❖ list[1]: This uses the typical array syntax to fetch the second item in the list.

  Los **nodeList** pueden ser vivos o muertos (no vivos o estáticos, si lo prefiere);

  - ✓ si son **vivos**, contendrán una referencia bidireccional con los nodos, de modo que cualquier modificación que realicemos en ellos se reflejará en la representación del nodo en el documento, y cualquier modificación que pudieran sufrir esos nodos por acción de otra instrucción se reflejaría en el nodeList (**incluso si se añaden o eliminan nodos**);
  - ✓ si son **muertos** la referencia también es bidireccional, pero sólo con los nodos que contenía el documento en el momento en el que fue creado el nodeList (al añadir o eliminar nodos posteriormente, el nodeList estático no será informado de este hecho).

  En la sección "Modificar el DOM" de este mismo tema se tratará de aclarar esta sutil diferencia entre nodeList vivo y muerto con un ejemplo.

- **Attribute**. Es un tipo de datos que hace referencia a un nodo de tipo atributo. No debemos confundirlo con el tipo attr, que es un objeto asociado a un nodo de tipo atributo (¿y cuál es la diferencia? attribute hereda de Node y consecuentemente utiliza las propiedades `nodeName` y `nodeValue`, mientras que attr es un objeto que usa las propiedades `name` y `value`).

- **NamedNodeMap**. Es una colección viva de nodos, similar a nodeList, pero en la que además de acceder a los nodos por su índice (que en este caso y a diferencia de nodeList no corresponde a ningún orden convenido), podemos hacerlo por su nombre a través del método `getNamedItem(`*`nombreDelNodo`*`)`.

- **HTMLCollection**. Es una colección viva y ordenada (como un nodeList) de nodos de tipo elemento en la que se puede acceder a sus elementos por el índice o por el id de ese elemento usando en ambos casos la sintaxis de corchetes.

**Attr**. Es un objeto asociado a un nodo de tipo atributo, pero en DOM 4 ya no hereda del objeto Node, de modo que no podemos utilizar `nodeName` ni `nodeValue`, pero en su lugar nos ofrece las propiedades `name` y `value`.

---

**Nota:** Por las limitaciones de este curso resulta imposible citar aquí las propiedades y métodos de todos los objetos del DOM, por lo que se recomienda acudir a alguna referencia (como https://developer.mozilla.org/en/Gecko_DOM_Reference) en caso de duda.

---

# *EL OBJETO "document"*

## *El objeto document* *(*/en-US/docs/Web/API/document)*

La raíz del árbol DOM de todas las páginas HTML es su objeto ***document***. A través de él podremos:

- Localizar y acceder a elementos concretos dentro del documento.
- Obtener información sobre los metadatos del documento.
- Conocer la ubicación del documento o cambiarla para redirigir al navegador/usuario a otro documento diferente.
- Leer y escribir cookies.
- Saber si el navegador ha terminado de cargar el contenido de la página y todos sus recursos (imágenes, códigos, elementos multimedia, …).

### Acceder y localizar elementos concretos dentro del documento

Básicamente existen 4 formas de acceder a un elemento de un documento HTML:

- **A través de una propiedad directa**. Algunos elementos únicos, como head y body, son accesibles directamente a través de las propiedades `head` y `body`, respectivamente, del objeto document.
- **A través de una colección (HTMLCollection)**. Para otros elementos que no tienen por qué ser únicos, el objeto document nos ofrece propiedades que devuelven una colección ordenada de esos elementos, como `forms`, `links`, `images`, `scripts`, `styleSheets`, y `frames` (para iFrames).
- **Explorando el árbol de nodos**. Utilizar las propiedades `childNodes`, `firstChild`, `lastChild`, `previousSibling`, `nextSibling`, y `parentNode`, procedentes de la interfaz nodo (al fin y al cabo el documento completo es también un nodo), para explorar los distintos elementos. Generalmente estas propiedades suelen utilizarse en combinación con `nodeType` y `nodeName` (ya tratadas en el tema anterior), o métodos como `hasChildNodes()`, que nos indica si el nodo contiene nodos anidados, para poder discriminar e identificar el nodo que nos interesa. Estas propiedades/métodos se describirán en la sección posterior de este mismo tema "Explorar el árbol DOM".
- **Buscándolo**. El objeto document nos ofrece los métodos:
    1. getElementById() (obsérvese que es Element y no Elements, pues no puede haber más de un elemento con el mismo id en un documento HTML), con él podemos buscar, el elemento cuyo id es uno concreto,
    2. getElementsByName(), con él podemos buscar los elementos que comparten un mismo valor del atributo name
    3. getElementsByTagName(), con él podemos buscar los elementos HTML de un tipo concreto (similar a las propiedades forms, images, scripts, …, pero devolviendo un nodeList en lugar de una HTMLCollection)

4. getElementsByClassName(), con él podemos buscar los elementos que pertenecen a una clase determinada.

---

**Nota:** Tras **guardar en una variable** un **nodeList** vivo generado con getElementsByTagName(), getElementsByClassName(), getElementsByName() con un determinado argumento, **no será necesario que volvamos a llamar a estos métodos con el mismo argumento nunca más**, pues siempre tendremos en esa variable una versión actualizada de los elementos que son identificados por ese argumento, aunque posteriormente a la asignación de la variable se creen o eliminen elementos del DOM.

---

- Búsqueda de elementos **usando como criterio de búsqueda selectores CSS**:

  Hay que recordar que los selectores CSS pueden ser de etiqueta, de id y de clase, y que se pueden agrupar con una coma, o combinar mediante los operadores de descendiente (espacio), hijo (>) y hermano menor (+).

  Para implementar este tipo de búsqueda el DOM tiene los siguientes métodos:

  - **querySelector(*cadenaSelector*)** devuelve el primer elemento que coincide con el selector.
  - **querySelectorAll(*cadenaSelector*)** devuelve un nodeList **muerto** de todos los elementos que coinciden con el selector.

  **Nota:**

  - Los métodos `querySelector()` y `querySelectorAll()` permiten utilizar todos los tipos de selectores de CSS que ya conocemos.
  - Selectores de pseudo-clase como **:hover** requieren una explicación adicional:
    - ✓ **CSS es un lenguaje declarativo**, es decir, las reglas de estilo configuradas están vigentes durante toda la vida del documento.
    - ✓ mientras que **JavaScript es un lenguaje ejecutado**, es decir, el selector entregado a los métodos `querySelector()` o `querySelectorAll()` se evalúa en **un instante**, devolviendo los nodos que en ese momento cumplen el criterio del selector.

    En otras palabras: si utilizamos el selector de pseudo-clase :hover con `querySelector()` o `querySelectorAll()` se seleccionarán los nodos sobre los que se encuentre el puntero en ese instante, no aquéllos sobre los que pueda colocarse el puntero posteriormente.

(Proponer ejercicio 2 de DOM.)

Un detalle interesante sobre la exploración y búsqueda de nodos de elemento es que los objetos de tipo `element` también ofrecen los mismos métodos de exploración y búsqueda, citados en el párrafo anterior, para el elemento **`document`** (excepto `getElementById()` porque sólo puede haber un elemento con un id determinado en todo el documento; no tendría sentido particularizarlo a nivel de elemento). ¿Qué importancia tiene esto? Mucha, pues nos **permite encadenar búsquedas**:

> **Por ejemplo**, podemos realizar una búsqueda sobre todo el documento para obtener el **nodeList** de los elementos de tipo table, y luego buscar dentro de la segunda de esas tablas los tr (*table row*, fila de tabla) que pertenezcan a la clase encabezado:
>
> ```
> document.getElementsByTagName('table')[1].querySelectorAll('tr.encabezado');.
> ```

En la sección posterior "Modificar el DOM" de este mismo tema encontrará un ejemplo sobre la concatenación de búsquedas.

### Obtener información sobre los metadatos del documento

Con las propiedades `characterSet` (sólo lectura) y `title` (lectura y escritura) del objeto **document** podemos conocer qué codificación de caracteres utiliza el documento y cuál es su título.

## La propiedad location

(Ya vista en temas anteriores)

## La propiedad readyState

La propiedad `readyState` nos permite saber en qué estado de carga se encuentra la página. Su valor es una cadena de texto que puede tener uno de estos 3 valores:

- ***loading***. Mientras el navegador está aún interpretando el documento HTML.
- ***interactive***. Cuando el navegador ya ha terminado de interpretar el documento HTML, pero aún no ha concluido la carga de otros recursos que utiliza la página, como imágenes.
- ***complete***. Cuando el navegador ha cargado definitivamente la página y todos sus recursos adicionales.

Por ejemplo:

```
001  <!DOCTYPE html>
002  <html>
003    <head>
004          <title>DOM</title>
005            <script>
006                  alert(document.readyState);
007            </script>
008    </head>
009    <body>
010       <button type='button' onclick='alert(document.readyState);'>Comprobar estado</button>
011    </body>
012  </html>
```

## *El objeto Element*

- Cada nodo de un documento tiene asociado un objeto de tipo `Node`.
  - si además es un nodo de elemento, la interfaz del objeto `Node` se complementará con la del objeto `Element`.
    - ➢ si además se trata de un elemento HTML (recuerde que el DOM es aplicable también a documentos XML), la interfaz del objeto `Element` se complementará con la del objeto `HTMLElement`.
      - ❖ y finalmente, dependiendo de qué tipo de elemento se trate, la interfaz de `HTMLElement` se complementará con la de un objeto más específico como `HTMLParagraphElement` o `HTMLImageElement`.

El resultado final es que **en un nodo de elemento** disponemos de una combinación de **propiedades y métodos** procedentes de diferentes interfaces, y gracias a ellos podemos:

- Obtener información sobre la posición del elemento.
- Gestionar los atributos del elemento.
- Explorar el árbol del DOM.
- Modificar el DOM del documento añadiendo, modificando o eliminando elementos.

---

**Nota:** Recuerde que en la sección anterior " Acceder y localizar elementos concretos dentro del documento" se explicó que el objeto element también ofrece los métodos de búsqueda de nodos `getElementsByTagName`, `getElementsByClassName`, `getElementsByName`, `querySelector` y `querySelectorAll`, de modo que **podemos concatenar unas búsquedas** con otras..

---

### Obtener información sobre la posición de un elemento

En la siguiente tabla se describen algunas propiedades que nos ofrecen información sobre la posición del elemento dentro de la página; todas estas propiedades son de sólo lectura.

| Propiedad | Descripción |
|---|---|
| offsetParent | Contiene una **referencia al elemento padre del actual**, calculándose las propiedades offsetTop y offsetLeft respecto a la frontera entre el borde y el **padding** (límite de relleno). |
| offsetHeight | **Contiene** la altura en píxeles del límite de borde de la caja del elemento; en otras palabras: **la distancia desde el exterior del borde superior al exterior del inferior**. |
| offsetWidth | **Contiene** la anchura en píxeles del límite de borde de la caja del elemento; en otras palabras: **la distancia desde el exterior del borde izquierdo al exterior del derecho**. |

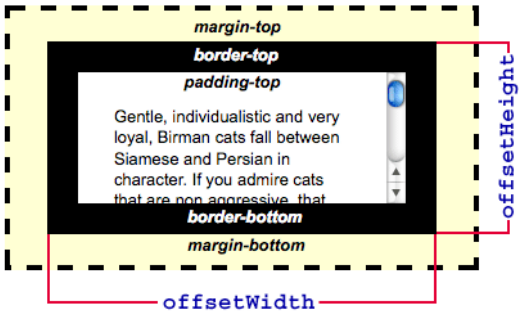
offsetTop **Contiene** la **distancia en píxeles entre la ordenada** del límite de padding superior del elemento padre y el límite de borde superior del elemento al que pertenece la propiedad.

offsetLeft **Contiene la distancia en píxeles entre la abscisa** del límite de padding izquierdo del elemento padre y el límite de borde izquierdo del elemento al que pertenece la propiedad.

Por ejemplo:

```
<!DOCTYPE html>
<html>
	<head>
		<title>DOM</title>
		<style>
			#exterior {
				position: absolute;
				width: 200px;
				height: 200px;
				top: 100px;
				left: 100px;
				background: red;
				padding: 25px;
			}
			#interior {
				margin: 50px;
				width: 100px;
				height: 100px;
				border: 1px solid black;
				background: yellow;
			}
		</style>
		<script>
			function informacion(){
				var divInterior = document.getElementById('interior');
				console.log('offsetParent: '+divInterior.offsetParent);
				console.log('offsetWidth: '+divInterior.offsetWidth);
				console.log('offsetHeight: '+divInterior.offsetHeight);
				console.log('offsetTop: '+divInterior.offsetTop);
				console.log('offsetLeft: '+divInterior.offsetLeft);
			}
		</script>
	</head>
	<body>
		<button onclick='informacion();'>INFORMACI&Oacute;N</button>
		<div id='exterior'>
			<div id='interior'></div>
		</div>
	</body>
</html>
```

Ejemplo:




El ejemplo de la imagen de arriba muestra una barra de desplazamiento y un offsetHeight que cabe en la ventana. Sin embargo, los elementos no desplazables podrían tener un offsetHeight mucho más grande que el área visible. Esos elementos están típicamente contenidos en elementos desplazables; con lo cual esos elementos no desplazables podrían ser parcial o completamente innvisibles, dependiendo de lo que se haya desplazado el elemento contenedor.

## Gestionar los atributos de un elemento

- A través del objeto **`Element`**, que posee propiedades asociadas a los principales atributos que puede tener un elemento genérico HTML, podremos controlar las propiedades:
  - `id`
  - `className`
  - `name`
  - `style`
  - y `title`.
- A través de los objetos de tipo **`HTMLxxxElement`**, podremos gestionar el resto de propiedades que puede tener cada tipo de elemento en particular.
- También podemos gestionar cualquier atributo a través de la propiedad `attributes` (de NamedNodemap) o de los métodos `getAttribute()`, `setAttribute()`, `removeAttribute()` y `hasAttribute()`.

> **Nota:** ¿Por qué molestarnos en aprender otras técnicas genéricas para gestionar los atributos si los elementos del DOM ya nos ofrecen acceso a ellos mediante propiedades? Porque recuerde que el DOM es aplicable también a documentos XML, y en estos los elementos y atributos no están predefinidos. Por ejemplo, si recibimos un documento XML mediante Ajax nos resultará muy cómodo poder acceder a sus elementos y atributos con los métodos del DOM.

En la siguiente tabla se describen las propiedades genéricas.

| Propiedad | Descripción |
|---|---|
| id | Lee o establece el atributo `id` del elemento. Recuerde que el id de cada elemento debe ser exclusivo (no puede haber dos elementos con el mismo id). |
| className | Lee o establece el atributo `class` del elemento. Pueden asignarse varias clases a un elemento separando sus nombres con espacios. |
| name | Lee o establece el atributo `name` del elemento. No confundir con nodeName, que lee el nombre del nodo (que en el caso de un nodo de elemento es la etiqueta HTML correspondiente). |
| title | Lee o establece el atributo `title` del elemento. El valor de este atributo se muestra como un mensaje de herramienta (*tooltip*) al colocar el puntero del ratón sobre el elemento. |
| style | Es una referencia a un objeto cuyas propiedades coinciden con las propiedades CSS configuradas para el elemento en su atributo `style` (recuerde del tema anterior que las declaraciones establecidas en el atributo `style` **poseen la máxima especificidad**). |

Por ejemplo, en el siguiente código tenemos un botón que cambia la clase del párrafo para aplicarle un estilo diferente y otro que actúa directamente sobre su atributo `style`.

```
001  <!DOCTYPE html>
002  <html>
003          <head>
004                  <title>DOM</title>
005                  <style>
006                          .claseParrafo {color:red;}
007                  </style>
008          </head>
009          <body>
010                  <p id='parrafo'>El estilo de este p&aacute;rrafo est&aacute;
     controlado por los botones.</p>
011                  <button type='button'
     onclick='document.getElementById("parrafo").className="claseParrafo";'>Cambiar
     clase</button>
012                  <button type='button'
     onclick='document.getElementById("parrafo").style.color="green";'>Cambiar atributo
     style</button>
013          </body>
014  </html>
```

Para cualquier otra propiedad (atributo) que no sea alguna de las referenciadas anteriormente, tendremos que recurrir a un procedimiento más general. En él disponemos de 2 opciones:

1. Partiendo del objeto Element, usar sus métodos:
   - `getAttribute(`*`nombre`*`)`, para leer el valor.
   - `setAttribute(`*`nombre`*`,`*`valor`*`)`, para crear o modificar el valor.
   - `removeAttribute(`*`nombre`*`)` para eliminar el valor
   - `hasAttribute(`*`nombre`*`)` averiguar si existe un atributo.

   Si intentamos obtener con `getAttribute(`*`nombre`*`)` un atributo que no existe obtendremos el valor `null`; es conveniente comprobar previamente la existencia del atributo con `hasAttribute(`*`nombre`*`)`. Por ejemplo:

```
001  <!DOCTYPE html>
002  <html>
003          <head>
004                  <title>DOM</title>
005                  <style>
006                          .claseParrafo {color:red;}
007                  </style>
008                  <script>
009                          function crearClase(){
010
          document.getElementById('parrafo').setAttribute('class','claseParrafo');
011                          }
012                          function leerClase(){
013
          if(document.getElementById('parrafo').hasAttribute('class')){
014
          alert(document.getElementById('parrafo').getAttribute('class'));
015                                  }else{
016                                          alert('No tiene clase');
017                                  }
018                          }
019                  </script>
020          </head>
021          <body>
022                  <p id='parrafo'>El estilo de este p&aacute;rrafo est&aacute;
     controlado por los botones.</p>
023                  <button type='button' onclick='crearClase();'>Crear clase</button>
024                  <button type='button' onclick='leerClase();'>Leer clase</button>
025          </body>
026  </html>
```

2. Partiendo de la colección viva de nodos NamedNodeMap, podremos usar la propiedad `attributes` que gestiona (lee y escribe) un tipo de dato compuesto por objetos de tipo `Attr`, es decir, por objetos con las propiedades `name` y `value`.

   En los namedNodeMap disponemos de los métodos:

   - `getNamedItem(`*`nombre`*`)` para obtener el atributo llamado *nombre*.
   - `setNamedItem(`*`objetoAttribute`*`)` para añadir o sustituir un atributo asignándole un objeto de tipo `Attribute`. *objetoAttribute* es el objeto de tipo Attribute devuelto por `createAttribute(`*`nombre`*`)`.
   - `removeNamedItem(`*`nombre`*`)` para eliminar el atributo llamado *nombre*.

   Generalmente el uso de `attributes` resulta un poco incómodo, salvo cuando lo que nos interese sea gestionar todos los atributos de un elemento.

   `Nota:` El objeto document nos proporciona el método `createAttribute(`*`nombre`*`)` para **crear un nodo de atributo**, obsérvese que *nombre* en este método es una cadena.

Por ejemplo:

```
001  <!DOCTYPE html>
002  <html>
003          <head>
004                  <title>DOM</title>
005                  <style>
006                          .claseParrafo {color:red;}
007                  </style>
008                  <script>
009                          function crearAtributo(){
010                                  atributo = document.createAttribute('class');
011                                  atributo.nodeValue = 'claseParrafo';
012
         document.getElementById('parrafo').attributes.setNamedItem(atributo);
013                          }
014                          function verAtributos(){
015                                  var i;
016                                  var respuesta = '';
017                                  var atributos =
     document.getElementById('parrafo').attributes;
018                                  for (i=0;i<atributos.length;i++){
019                                          respuesta += atributos[i].name + '=' +
     atributos[i].value + '\n';
020                                  }
021                                  alert(respuesta);
022                          }
023                  </script>
024          </head>
025          <body>
026                  <p id='parrafo'>El estilo de este p&aacute;rrafo est&aacute;
     controlado por los botones.</p>
027                  <button type='button' onclick='crearAtributo();'>Cambiar
     clase</button>
028                  <button type='button' onclick='verAtributos();'>Ver todos los
     atributos</button>
029          </body>
030  </html>
```